

NYAHURURU COUNTY REFERRAL HOSPITAL

(NCRH)

IT HANDOVER & TECHNICAL DOCUMENTATION

Version 1.0 · August 2026

1. Executive Summary

This document is the complete technical and operational handover package for the NCRH Digital Platform — a custom-built web application that serves as the primary digital interface for the Nyahururu County Referral Hospital. It is intended for the NCRH IT Department to assume ownership, day-to-day administration, and ongoing maintenance of the system.

What the System Is

The NCRH Digital Platform is a full-stack web application comprising two parts:

- A public-facing hospital website providing health services information, doctor profiles, appointment booking, blood donation registration, career listings, news and events, tender listings, and anonymous feedback.
- A secure administrative dashboard and a separate research management portal used by hospital staff, researchers, peer reviewers, and the ethics committee.

Technology at a Glance

Component	Technology	Notes
Frontend (UI)	React 19 + Vite + Tailwind CSS	Single-page application; deployed as static files
Backend (API)	Node.js + Express 5	RESTful JSON API; runs as a long-lived Node process
Primary Database	MySQL (via Sequelize ORM)	Migrated from MongoDB; schema managed with sequelize-cli
Real-time	Socket.IO	Live dashboard notifications
File Storage	Local disk (server /uploads)	Must be backed up; consider cloud storage for production
Payments	M-Pesa STK Push (Safaricom Daraja)	Research proposal submission fees
Email	Nodemailer + Gmail SMTP	Transactional emails: verification, password reset, notifications
PDF Generation	PDFKit + pdf-lib	Certificate and decision letter generation

Handover Scope

This document covers: system architecture, all environment variables and their purpose, database structure, user roles and permissions, API surface, background jobs, file upload conventions, deployment procedure, day-to-day administrative tasks, known limitations, and recommended next steps.

2. System Architecture

2.1 High-Level Overview

The platform follows a classic three-tier architecture: a React single-page application (SPA) communicates with an Express REST API over HTTPS, which in turn reads from and writes to a MySQL database. A Socket.IO layer provides real-time push notifications to the dashboard.

Tier	Role	Technology
Presentation	Browser-rendered UI; communicates only with the API	React 19 / Vite / Tailwind CSS
Application	Business logic, authentication, file handling, email, PDF	Node.js / Express 5
Data	Relational persistence; all state lives here	MySQL 8 / Sequelize ORM

2.2 Repository Structure

Server (Backend)

```
server/
  app.js           ← Express app setup, middleware, routes
  server.js        ← HTTP server bootstrap, DB connect, cron jobs
  constants/
    authConfig.js  ← bcrypt rounds, lockout config
    researchIndex.js ← All research domain enums and constants
  controllers/     ← One file per domain (request → response logic)
  middleware/
    auth.js        ← JWT verification, role gates
    upload.js      ← Multer config + magic-byte file validation
  routes/          ← Express routers (one per domain)
  services/        ← Business logic called by controllers
  utils/           ← Shared utilities (email, logger, validators, etc.)
  scripts/         ← Standalone maintenance scripts & cron jobs
  sequelize/
    config/config.js ← DB connection config (reads from .env)
    models/         ← 30 Sequelize model definitions
    migrations/     ← sequelize-cli DDL migrations
    seeders/        ← (empty; use seeder.js for superadmin seed)
    uploads/        ← File upload storage (must be backed up)
```

Client (Frontend)

```
client/
  src/
    App.jsx        ← Root router; all routes defined here
    config/
      env.js       ← VITE_API_URL read here; only config file needed
      dashboardRoles.js ← Maps dashboard routes to allowed staff roles
    api/           ← Axios instances and per-domain API call functions
    pages/         ← Top-level page components (public site)
    components/
      Dashboard/   ← Staff dashboard pages
      research/    ← Research portal (researcher/reviewer/committee)
      common/      ← Shared UI primitives (Button, Modal, DataTable...)
```

```
public/  
  _redirects      ← Netlify/Vercel SPA fallback rule  
vite.config.js    ← Build config; dev proxy to localhost:5000
```

3. Environment Variables

All sensitive configuration is loaded from environment variables. Never commit these to source control. Create a .env file in the server root directory.

Security Note

The server validates required variables at startup and will refuse to start if any are missing. Never share these values over email or chat. Store them in a password manager or secrets vault.

3.1 Server Environment Variables (.env)

Core Application

Variable	Required	Example Value	Purpose
NODE_ENV	Yes	production	Controls logging verbosity, error detail, CSP. Must be 'production' in live.
PORT	Yes	5000	TCP port the server listens on. Nginx proxies to this.
JWT_SECRET	Yes	<random 64-char string>	Signs access tokens. Changing this invalidates all sessions.
JWT_REFRESH_SECRET	Yes	<different 64-char string>	Signs refresh tokens. Must differ from JWT_SECRET.
CORS_ORIGINS	Yes	https://ncrh.go.ke	Comma-separated list of allowed frontend origins. No trailing slash.
FRONTEND_URL	Yes	https://ncrh.go.ke	Used in email links (password reset, verification).
BCRYPT_SALT_ROUNDS	No	12	bcrypt work factor. 12 is the recommended minimum. Higher = slower login.
LOG_LEVEL	No	info	Pino log level: trace, debug, info, warn, error, fatal.

Database (MySQL)

Variable	Required	Example Value	Purpose
DB_HOST	Yes	localhost	MySQL server hostname or IP.
DB_PORT	Yes	3306	MySQL port (3306 default).
DB_NAME	Yes	ncrh_db	Database name.
DB_USER	Yes	ncrh_user	MySQL user account.
DB_PASSWORD	Yes	<strong password>	Database password. Cannot be empty in production.
DB_SSL	Yes (prod)	true	Must be 'true' in production. Enforces TLS to MySQL.

Variable	Required	Example Value	Purpose
DB_SSL_REJECT_UNAUTHORIZED	No	true	Set false only if using self-signed cert (not recommended).
DB_POOL_MAX	No	10	Max connections in pool.
DB_POOL_MIN	No	0	Min connections in pool.
DB_POOL_ACQUIRE_MS	No	30000	Max ms to wait for a connection.
DB_POOL_IDLE_MS	No	10000	How long idle connections are kept before release.
DB_TIMEZONE	No	+03:00	MySQL timezone. Set to East Africa Time (+03:00).
DB_LOGGING	No	false	Set 'true' to log all SQL queries (development only).

Email (Nodemailer)

Variable	Required	Example Value	Purpose
EMAIL_SERVICE	No	gmail	Nodemailer service name. 'gmail' for Gmail; omit for custom SMTP.
EMAIL_USER	Yes	noreply@ncrh.go.ke	Sender email address.
EMAIL_PASSWORD	Yes	<app password>	Gmail App Password (not your account password). Enable 2FA first.

M-Pesa (Safaricom Daraja)

Variable	Required	Example Value	Purpose
MPESA_CONSUMER_KEY	Yes*	<from Daraja portal>	API consumer key for your M-Pesa app.
MPESA_CONSUMER_SECRET	Yes*	<from Daraja portal>	API consumer secret.
MPESA_SHORTCODE	Yes*	174379	Business shortcode (174379 = sandbox). Change for production.
MPESA_PASSKEY	Yes*	<from Daraja portal>	Lipa Na M-Pesa passkey.
MPESA_API_URL	No	https://api.safaricom.co.ke	Sandbox: sandbox.safaricom.co.ke. Change for production.
MPESA_CALLBACK_URL	Yes*	https://ncrh.go.ke/api/payments/mpesa/callback	Must be a public HTTPS URL reachable by Safaricom.
USE_PRODUCTION_AMOUNTS	No	true	Set 'true' in production. In dev/test, fee is KES 1.

* Required only if M-Pesa payment processing is active.

Research & Scheduling

Variable	Required	Example Value	Purpose
CERTIFICATE_SIGNING_SECRET	Yes	<random 64-char string>	HMAC-SHA256 key for QR certificate tokens. Must never change once certs are issued.
RESEARCH_EXPIRY_CRON_ENABLED	No	true	Set 'false' to disable the built-in daily expiry cron (use external cron instead).
REVIEWER_REMINDER_CRON_ENABLED	No	true	Set 'false' to disable the built-in daily reviewer reminder cron.

First-Run Seeding

Variable	Required	Example Value	Purpose
SEED_SUPERADMIN_EMAIL	No	admin@ncrh.go.ke	Email for the initial superadmin account (seeder.js).
SEED_SUPERADMIN_PASSWORD	Yes (production seed)	<strong temp password>	Temporary password. Must be set in production; default is rejected by seeder.

3.2 Client Environment Variables (.env in client/)

Variable	Required	Example Value	Purpose
VITE_API_URL	Yes	https://ncrh.go.ke/api	Full URL to the backend API. Must match CORS_ORIGINS on server.

Generating Secrets
Use this command to generate cryptographically secure random strings: `node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"`

4. Database

4.1 Overview

The system uses MySQL 8. The schema is managed by Sequelize ORM with version-controlled migrations (sequelize-cli). All schema changes must go through migrations — never alter the database manually.

Property	Value
Engine	MySQL 8.0+
ORM	Sequelize v6
Charset	utf8mb4 / utf8mb4_unicode_ci
Schema management	sequelize-cli (npm run db:migrate)
Primary keys	BIGINT UNSIGNED AUTO_INCREMENT on all tables
Column naming	snake_case in MySQL; camelCase in JS models (underscored: true)
Timestamps	created_at / updated_at on all tables (managed by Sequelize)
Money columns	DECIMAL(n,2) — never FLOAT or DOUBLE

4.2 Database Tables

Authentication & Users

Table	Purpose	Key Columns
users	Staff accounts (admin, doctor, IT, comms, research, vendor, etc.)	id, email, password (bcrypt), role, employee_id, failed_login_attempts, lock_until
profiles	Extended staff profile data	id, user_id (FK→users), avatar, bio, social_links
token_blacklist	Revoked JWT JTI values (logout / password change)	id, jti, expires_at
researchers	Researcher/reviewer/committee accounts (separate auth namespace)	id, email, password (bcrypt), role, status, is_committee

Research Domain

Table	Purpose	Key Columns
submissions	All submission types (proposals, amendments, continuingreviews, closures)	id, research_id, researcher_id, submission_type, status, seru_number, approval_valid_until
continuing_review_details	Progress report payload for continuing_review submissions (1:1)	id, submission_id (FK→submissions), progress_summary, documents (JSON)
closure_details	Closeout payload for study_closure submissions (1:1)	id, submission_id (FK→submissions), closure_reason, closeout_report_file
reviews	Individual reviewer and committee review records	id, research_id, reviewer_id, stage, decision, round, is_latest

Table	Purpose	Key Columns
research_reviewers	Reviewer-to-submission assignment tracking	id, research_id, reviewer_id, review_status, first_reminder_sent_at
research_decision_reports	Compiled decision reports (RO-controlled; not shown to researcher until released)	id, research_id, round, reviewer_comments (JSON), final_decision, released_at
co_investigator_assignments	Co-investigator access grants per study	id, research_id, researcher_id, can_edit, accepted_at
protocol_deviations	Adverse events / deviations reported on approved studies	id, research_id, deviation_type, severity, status, deadline
counters	Atomic sequence counters (research IDs, tender numbers, SERU numbers)	id, key, value
payments	M-Pesa STK push payment records for proposals	id, research_id, amount (DECIMAL), status, checkout_request_id
certificates	Clearance and completion certificates with QR codes	id, research_id, type, certificate_number, sign_token, qr_code_data_url (MEDIUMTEXT)

Medical Services

Table	Purpose
appointments	Named patient appointment bookings
anonymous_appointments	Privacy-first appointment requests (case_code instead of name)
doctors	Doctor profiles linked to user accounts (1:1)
blood_donors	Blood donor registry
urgent_blood_requests	Public blood donation urgent requests
ambulance_bookings	Ambulance dispatch requests

Operations & Content

Table	Purpose
inventory	Hospital supply / medication inventory with expiry tracking
vehicles	Ambulance and hospital vehicle fleet
tenders	Procurement tender listings
bids	Vendor bids on tenders
fraud_reports	Anonymous fraud / misconduct reports
news	News articles published on the public site
notices	Announcements (scheduled visibility with start/end dates)
gallery	Photo and video gallery items
gallery_categories	Categories for gallery items

Table	Purpose
events	Hospital events
services	Hospital service definitions with icons and descriptions
feedback	Public feedback submissions (name and email optional)
reports	Downloadable institutional reports and publications
audit_logs	Security-sensitive action audit trail

4.3 Database Migrations

Migrations live in `server/sequelize/migrations/`. Run them with:

```
# Apply all pending migrations
npm run db:migrate

# Check migration status
npm run db:migrate:status

# Rollback the last migration
npm run db:migrate:undo

# Rollback ALL migrations (destructive — drops all tables)
npm run db:migrate:undo:all
```

Current Migrations (in order)

Migration File	What It Does
20260722000000-baseline-schema.js	Creates all original tables: users, profiles, token_blacklist, researchers, submissions (as 'researches'), reviews, research_reviewers, payments, certificates, tenders, bids, inventory, vehicles, news, notices, gallery, events, services, feedback, reports, doctors, appointments, blood_donors, ambulance_bookings, fraud_reports, audit_logs, counters, protocol_deviations, co_investigator_assignments, research_decision_reports
20260805120000-seru-number-non-unique.js	Drops the UNIQUE constraint on seru_number (a SERU number belongs to a study, shared by all child submissions)
20260806090000-drop-submission-stage-blobs.js	Drops sparse JSON blob columns from submissions (continuing_review_data, progress_data, closure_report, etc.) — replaced by 1:1 detail tables
20260806090100-rename-researches-to-submissions.js	Renames the 'researches' table to 'submissions' (MySQL cascades the rename to all FK constraints)
20260806090200-create-continuing-review-details.js	Creates continuing_review_details table (1:1 with submissions)
20260806090300-create-closure-details.js	Creates closure_details table (1:1 with submissions)

4.4 Initial Database Bootstrap

After running migrations, seed the first superadmin account:

```
# Set credentials in .env first, then:
SEED_SUPERADMIN_EMAIL=admin@ncrh.go.ke
SEED_SUPERADMIN_PASSWORD=<strong temp password>

node seeder.js

# The account is created with mustChangePassword=true
# The admin MUST change the password on first login
```

5. User Roles & Permissions

5.1 Staff Roles (users table)

Staff accounts are managed by the superadmin and admin. Registration requires an existing admin account — self-registration is not available.

Role	Dashboard Access	Key Permissions
admin	Full operational access	Manage appointments, tenders, inventory, vehicles, news, notices, gallery, events, feedback, fraud reports, blood donors, careers, services, research admin, users
it	Technical administration	User management, audit logs, all content management, system configuration
research	Research portal administration	Manage research submissions, assign reviewers, release decision reports, issue certificates, invite reviewers & committee members
doctor	Medical operations	View own appointments, update own doctor profile and availability
communication	Content & communications	Manage news, events, gallery, notices, feedback, fraud reports

5.2 Researcher Roles (researchers table)

The research portal has its own authentication namespace. Researchers self-register at </research/register> and verify their email before they can submit proposals.

Role	Who They Are	What They Can Do
researcher	External researchers submitting proposals	Register; submit initial proposals, amendments, continuing reviews, and study closures; track submission status; download certificates; pay submission fees via M-Pesa
co_investigator	Named collaborator on a study	View and optionally edit studies they have been invited to co-investigate
reviewer	External peer reviewer	Review assigned submissions; submit scored reviews with feedback; view own review history
research_committee	Ethics committee member	Vote on proposals at committee stage; view all proposals in the committee queue; access final approval feed

Reviewer & Committee Account Creation

Reviewers and committee members are NOT self-registered. The Research Officer (staff with 'research' role) invites them via the admin dashboard. They receive an email with a one-time link to set their password. The Research Officer can revoke access at any time.

6. API Surface

6.1 Base URL

```
Production: https://ncrh.go.ke/api
Development: http://localhost:5000/api
```

6.2 Authentication

The API uses JWT Bearer tokens. Include the token in every protected request:

```
Authorization: Bearer <access_token>
```

Tokens can also be sent as a cookie (jwt) or query parameter (?token=) for specific use cases (file downloads).

Endpoint	Method	Auth	Purpose
POST /api/auth/login	POST	Public	Staff login → returns access_token + refresh_token
POST /api/auth/logout	POST	Staff	Blacklists the current token JTI
POST /api/auth/refresh	POST	Public (cookie)	Exchange refresh_token for a new access_token
GET /api/auth/me	GET	Staff	Returns current logged-in staff user
POST /api/auth/register	POST	Admin/IT	Create a new staff user account
POST /api/researchers/register	POST	Public	Researcher self-registration
POST /api/researchers/verify-email	POST	Public	Email verification after registration
POST /api/researchers/login	POST	Public	Researcher login
POST /api/researchers/forgot-password	POST	Public	Request password reset link
POST /api/researchers/reset-password	POST	Public	Complete password reset
GET /api/researchers/me	GET	Researcher	Get own researcher profile
POST /api/reviewers/set-password	POST	Public (token)	Invited reviewer sets initial password

6.3 Route Groups

Route Prefix	Domain	Primary Auth	Notes
/api/auth	Authentication	Public / Staff	Rate-limited: 15 req/hour
/api/users	Staff user management	Admin/IT	CRUD on staff accounts
/api/profile	Staff profile	Staff	Self-service profile update, password change
/api/researchers	Researcher auth	Public / Researcher	Rate-limited: 15 req/hour
/api/reviewers	Reviewer management	Research Admin	Invite, revoke, manage reviewers and

Route Prefix	Domain	Primary Auth	Notes
			committee
/api/research	Research submissions	Researcher/Reviewer/Admin	Full submission lifecycle
/api/payments	M-Pesa payments	Researcher/Admin	STK push, callback, verification
/api/certificates	Ethics certificates	Researcher/Admin	Issue, verify, list certificates
/api/appointments	Appointments	Public/Staff	Book and manage appointments
/api/anonymous	Anonymous appointments	Public/Staff	Privacy-first appointments
/api/ambulance-bookings	Ambulance dispatch	Public/Staff	Ambulance request and tracking
/api/doctors	Doctor profiles	Public/Doctor	Doctor directory and availability
/api/blood-donation	Blood donor registry	Public/Staff	Donor registration and management
/api/urgent-request	Urgent blood requests	Public/Staff	Emergency blood type requests
/api/inventory	Hospital inventory	Admin/IT	Stock management with expiry tracking
/api/vehicles	Fleet management	Admin/IT	Vehicle registry and maintenance tracking
/api/tenders	Tender listings	Public/Admin	Procurement tender management
/api/bids	Tender bids	Vendor/Admin	Bid submission and evaluation
/api/fraud	Fraud reports	Public/Admin	Anonymous misconduct reports
/api/news	News articles	Public/Staff	News CMS
/api/notices	Announcements	Public/Staff	Notice board with scheduling
/api/events	Events	Public/Staff	Hospital events calendar
/api/gallery	Media gallery	Public/Staff	Photo and video gallery
/api/services	Hospital services	Public/Staff	Service definitions
/api/feedback	Public feedback	Public/Staff	Feedback form with admin response
/api/reports	Institutional reports	Public/Admin	Downloadable reports library
/api/audit-logs	Audit trail	Superadmin/IT	Security event log

6.4 Health Check

```
GET /health
```

Response:

```
{ "success": true, "message": "NCRH API is running", "environment": "production",
"timestamp": "..." }
```

7. Research Management System

The research module is the most complex part of the platform. It implements the full KEMRI SERU (Scientific and Ethics Review Unit) research ethics workflow.

7.1 Submission Types

Type	Code	Description	ID Format
Initial Proposal	initial_proposal	New research study proposal	NCRH-{YYYY}-{NNNNN}
Amendment	amendment	Protocol change to an approved study	<parent>-AM{N}
Continuing Review	continuing_review	Annual progress report (SERU Type B)	<parent>-CR{N}
Study Closure	study_closure	Closeout report when study ends	<parent>-CLO

7.2 Submission Lifecycle (Initial Proposal)

Status transitions for an initial proposal:

```
draft
→ awaiting_payment      (researcher submits; payment required)
→ submitted             (payment confirmed)
→ returned_for_correction (admin returns for admin-completeness issues)
→ under_review          (admin assigns reviewers; 2-5 reviewers)
→ revision_requested     (reviewers request changes)
→ pending_committee_review (all reviewers submitted; committee quorum check)
→ pending_officer_review (committee voted; RO compiles decision report)
→ approved / rejected / suspended
→ expired               (auto-set by daily cron after approvalValidUntil)
→ closed                (study closure approved)
```

7.3 Review Types

Type	Who Reviews	Notes
Full Committee	2–5 external reviewers + committee vote	Default for all initial proposals
Expedited	Research Officer only	For low-risk / observational studies
Secretariat	Research Officer only	Administrative review for minor amendments

7.4 Certificate System

Two certificate types are issued automatically when a study reaches the appropriate stage:

Certificate	Trigger	Format	Verification
Ethics Clearance (Proposal Approval)	Study status → approved	PDF with QR code	GET /api/certificates/verify? token=<token>&number=<cert_numbe r>

Certificate	Trigger	Format	Verification
Completion Certificate	Study closure approved	PDF with QR code	Same verification endpoint

Certificates include a HMAC-SHA256 sign token truncated to 24 hex characters. The verification endpoint uses `crypto.timingSafeEqual` for timing-attack resistance. The `CERTIFICATE_SIGNING_SECRET` must never change after certificates are issued — doing so permanently breaks all QR code verification.

7.5 M-Pesa Payment Flow

1. Researcher initiates payment → `POST /api/payments/initiate`
2. Server sends STK Push to researcher's phone via Safaricom Daraja API
3. Researcher approves on phone
4. Safaricom sends callback → `POST /api/payments/mpesa/callback`
5. Server updates payment status and moves submission to 'submitted'
6. Frontend polls `GET /api/payments/verify/:checkoutRequestId` until resolved

M-Pesa Production Checklist

1. Switch `MPESA_API_URL` to `https://api.safaricom.co.ke`
2. Replace sandbox shortcode, passkey, consumer key/secret with live credentials
3. Set `USE_PRODUCTION_AMOUNTS=true` (production fee: KES _____)
4. `MPESA_CALLBACK_URL` must be a public HTTPS endpoint reachable by Safaricom servers

8. Background Jobs & Scheduled Tasks

8.1 Built-in Cron Jobs

Two cron jobs run inside the server process (via node-cron). Both can be disabled if you prefer external scheduling (e.g. Linux crontab or cloud scheduler).

Job	Schedule	What It Does	Disable Flag
Research Expiry Job	Daily at 07:00 (Africa/Nairobi)	• Marks approved studies as expired if approvalValidUntil has passed • Creates a protocol deviation record for expired studies • Emails researchers 28-day and 42-day (investigational product) renewal reminders	RESEARCH_EXPIRY_CRON_ENABLED=false
Reviewer Reminder Job	Daily at 08:00 (Africa/Nairobi)	• Finds reviewer assignments pending for 7+ days with no reminder sent • Emails the reviewer once • Records firstReminderSentAt to prevent duplicate emails	REVIEWER_REMINDER_CRON_ENABLED=false

8.2 Manual Maintenance Scripts

Script	Run Command	Purpose	Frequency
Purge Expired Tokens	npm run maintenance:purge-tokens	Deletes expired rows from token_blacklist table. Keeps the table lean.	Weekly (add to crontab)
Backfill SERU Numbers	node scripts/backfillSeruNumbers.js	One-time script to assign SERU numbers to studies approved before the automation was added.	Run once only

8.3 Recommended Crontab (Linux)

```
# /etc/cron.d/ncrh (run as the node service user)

# Purge expired JWT blacklist tokens — weekly Sunday 02:00
0 2 * * 0 ncrh cd /opt/ncrh/server && node scripts/purgeExpiredTokens.js >>
/var/log/ncrh/purge.log 2>&1

# Database backup — daily 01:00
0 1 * * * ncrh /opt/ncrh/scripts/db-backup.sh >> /var/log/ncrh/backup.log 2>&1
```

9. File Storage

9.1 Overview

The system stores user-uploaded files (research proposals, certificates, gallery images, tender documents, etc.). In the original configuration these were written to the server's local disk under uploads/. This section documents the recommended approach for cPanel hosting: keeping files on cPanel's File Manager (simple, no code changes) OR connecting to a cloud storage bucket (recommended for production reliability).

9.2 Option A — cPanel File Manager (No Code Changes Required)

This is the simplest path. The Node.js app writes files to the uploads/ folder inside your application directory on the cPanel server. cPanel's File Manager gives you a browser-based interface to view, download, and delete those files.

How it Works on cPanel

What	Where on cPanel Server
Application root	e.g. /home/<cpanel_user>/ncrh-server/
Uploads folder	/home/<cpanel_user>/ncrh-server/uploads/
View files	cPanel → File Manager → navigate to the uploads folder
Download a file	Select file in File Manager → click Download
Delete a file	Select file → click Delete
Check disk usage	cPanel → Disk Usage widget on the main cPanel dashboard

Backup Uploads via cPanel

cPanel provides two built-in backup tools:

- cPanel → Backup Wizard → Full Backup — downloads a complete.tar.gz of your home directory, which includes uploads/. Run this weekly and store the archive offsite.

- cPanel → Backup → Download a Home Directory Backup — same archive, faster to access.
- If your host provides JetBackup or R1Soft, use those for automated daily backups.

10. Security Architecture

10.1 Authentication

Mechanism	Detail
JWT Access Tokens	HS256-signed; short-lived; verified on every request
Refresh Tokens	Long-lived; stored in httpOnly cookie; rotate on every use
Token Blacklist	Logout and password change immediately blacklist the JWT JTI in the database
Password Hashing	bcrypt with configurable work factor (default 12)
Account Lockout	5 failed login attempts → 15-minute lockout; configurable in constants/authConfig.js
Email Verification	Required before researcher accounts are active
mustChangePassword flag	Set on seeded / admin-created accounts; blocks all requests except /change-password
Two Auth Namespaces	Staff (users table) and Researchers (researchers table) are completely separate; cross-namespace tokens are rejected at the middleware

10.2 Middleware Security Layers

Middleware	What It Does
helmet	Sets 17 HTTP security headers (HSTS, X-Frame-Options, X-Content-Type-Options, etc.)
CORS	Strict allowlist; only CORS_ORIGINS may make cross-origin requests
HPP	HTTP Parameter Pollution protection; prevents duplicate query parameter attacks
express-rate-limit	Global: 200 req/15min. Auth/researcher endpoints: 15 req/hour
Body size limits	JSON and URL-encoded bodies limited to 10kb; file sizes limited per upload context
CSP	Content Security Policy enabled in production (built dynamically from CORS_ORIGINS)
dotfiles: deny	Static file serving blocks access to dotfiles (.env, .git, etc.)

10.3 Input Validation

All API inputs are validated using Zod schemas before reaching controllers. Key protections:

- Email addresses are normalised to lowercase
- Phone numbers are normalised to the E.164 format (254XXXXXXXX)
- Passwords: minimum 8 characters, maximum 128 characters
- Pagination: page ≥ 1 , limit 1–50 (capped at 50)
- Unexpected fields are stripped; unrecognised query parameters cause 400 responses

10.4 Audit Logging

Security-sensitive actions are logged to the audit_logs table. Logged events include:

- User account creation, modification, deletion

- Role changes
- Failed login attempts and account lockouts
- Password changes
- Admin actions (award tender, issue certificate, release decision report)

Audit logs are visible only to IT roles in the dashboard.

11. Deployment

11.1 Prerequisites

Requirement	Minimum Version	Notes
Node.js	18 LTS (20 LTS recommended)	Use nvm or fnm to manage Node versions
npm	9+	Bundled with Node.js
MySQL	8.0+	Create a dedicated database and user; enable SSL in production
Certbot / Let's Encrypt	Any current	Free SSL certificates for the domain
PM2	5+	Process manager to keep the Node server running and auto-restart on crash

11.2 First-Time Server Setup

```
# 1. Install Node.js (using nvm)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
nvm install 20
nvm use 20

# 2. Install PM2 globally
npm install -g pm2

# 3. Clone / copy the project
mkdir -p /opt/ncrh && cd /opt/ncrh
# (copy server and client directories here)

# 4. Install server dependencies
cd /opt/ncrh/server && npm install --omit=dev

# 5. Create .env (copy .env.example, fill in all values)
cp .env.example .env && nano .env

# 6. Run database migrations
npm run db:migrate

# 7. Seed initial superadmin
node seeder.js

# 8. Start server with PM2
pm2 start server.js --name ncrh-api
pm2 save
pm2 startup # follow the printed command to auto-start on reboot
```

11.3 Frontend Build & Deploy

```
# In client directory
cd /opt/ncrh/client
```

```
# Install dependencies
npm install

# Set production API URL
echo 'VITE_API_URL=https://ncrh.go.ke/api' > .env

# Build the SPA
npm run build

# Output is in client/dist/
# Copy dist/ contents to Nginx web root
cp -r dist/* /var/www/ncrh/
```

11.4 PM2 Commands

Command	Purpose
pm2 status	Show running processes and their status
pm2 logs ncrh-api	Stream live logs from the API server
pm2 logs ncrh-api --lines 200	Show last 200 log lines
pm2 restart ncrh-api	Restart the API server (zero-downtime with --reload)
pm2 reload ncrh-api	Zero-downtime reload (sends SIGINT, waits for graceful shutdown)
pm2 stop ncrh-api	Stop the server
pm2 monit	Real-time dashboard of CPU, memory, logs

12. Day-to-Day Administration

12.1 User Account Management

Task	Where	Who Can Do It
Create a new staff user	Dashboard → Users → Add User	Admin, IT, Superadmin
Deactivate a staff account	Dashboard → Users → Edit → Toggle Active	Admin, IT, Superadmin
Reset a staff user's password	Dashboard → Users → Edit → Reset Password	Admin, IT
Change own password	Dashboard → Profile → Change Password	Any logged-in staff
Invite a reviewer	Dashboard → Research → Reviewers → Invite	Research Officer, Admin, Superadmin
Revoke reviewer access	Dashboard → Research → Reviewers → Revoke	Research Officer, Admin, Superadmin
Grant committee access	Dashboard → Research → Reviewers → Grant Committee	Research Officer, Admin, Superadmin
Manage researcher accounts	Dashboard → Research → (admin view)	Research Officer, Admin

12.2 Content Management

Content Type	Dashboard Path	Notes
News articles	Dashboard → News	Supports draft/published state; image upload
Notices	Dashboard → Notices	Scheduled visibility (start date / end date); duplicate feature
Events	Dashboard → Events	Date-based events with image
Gallery	Dashboard → Gallery	Images and videos up to 100 MB; category management
Services	Dashboard → Services	Hospital service listings with icon images
Tenders	Dashboard → Tenders	Full procurement lifecycle; award, extend deadline, close
Reports (downloads)	Dashboard → Reports	Institutional PDF reports available for public download
Doctors	Dashboard → (doctor role login)	Doctor manages own profile and availability

12.3 Research Administration

Task	Action
View all submissions	Dashboard → Research → All Submissions
Return a submission for correction	Open submission → Return for Correction → add reason
Assign reviewers to a submission	Open submission → Assign Reviewers (select 2–5 from reviewer pool)

Task	Action
View committee queue	Research Portal → Committee Dashboard → Committee Queue
Compile decision report	Open submission → Decision Report → edit committee comment → set final decision
Release decision report to researcher	Decision Report → Release (researcher can now see outcome)
Issue ethics clearance certificate	Open approved submission → Issue Certificate
Verify a certificate	GET /api/certificates/verify?token=<token>&number=<cert_number> (public endpoint)
View revenue	Dashboard → Research → Payments → Revenue Summary
Refund a payment	Dashboard → Research → Payments → Refund (admin only)

12.4 Monitoring

What to Monitor	How	Alert Threshold
Server health	GET https://ncrh.go.ke/health	Non-200 response = server down
Process status	pm2 status	Status not 'online' = restart needed
Disk usage (uploads)	df -h /opt/ncrh/server/uploads	>80% = expand or migrate to cloud storage
Database connections	MySQL SHOW PROCESSLIST	Connections near pool max = investigate slow queries
Email deliverability	Check spam folder for verification emails	Bounce rate >5% = check Gmail app password / SMTP config
Cron job logs	pm2 logs ncrh-api grep 'Expiry job\ Reminder job'	'failed' in output = investigate
Audit logs	Dashboard → Audit Logs	Review weekly; investigate unexpected admin actions

13. Backup & Recovery

13.1 What Must Be Backed Up

Asset	Why Critical	Backup Strategy
MySQL database	All research submissions, user accounts, payments, certificates, audit trail	Daily mysqldump; retain 30 days; replicate offsite
uploads/ directory	All PDFs, images, videos, certificates — loss is permanent	Daily rsync to offsite location or cloud bucket
.env file	All secrets; loss means re-rotating all keys and re-issuing all tokens	Store encrypted in a password vault; document all values offline
CERTIFICATE_SIGNING_SECRET	Loss permanently breaks all issued certificate QR codes	Store in at least two separate offline locations
Nginx config	Hours of setup if server needs to be rebuilt	Version-control or backup with server config

13.2 Recovery Procedure

In the event of database loss:

- 1. Restore the most recent .sql.gz backup
- 2. gunzip ncrh_<date>.sql.gz | mysql -u \$DB_USER -p \$DB_NAME
- 3. Run npm run db:migrate to apply any pending migrations
- 4. Restore the uploads/ directory from backup
- 5. Restart the server (pm2 restart ncrh-api)
- 6. Verify health endpoint returns 200

14. Dependency Inventory

14.1 Server Dependencies

Package	Version	Purpose
express	^5.1.0	HTTP framework
sequelize	^6.37.8	ORM for MySQL
mysql2	^3.15.3	MySQL driver for Sequelize
jsonwebtoken	^9.0.2	JWT sign/verify
bcryptjs	^3.0.3	Password hashing
helmet	^8.2.0	HTTP security headers
cors	^2.8.5	Cross-Origin Resource Sharing
express-rate-limit	^8.5.2	Rate limiting
hpp	^0.2.3	HTTP Parameter Pollution protection
multer	^2.0.2	File upload handling
compression	^1.8.1	gzip response compression
cookie-parser	^1.4.7	Cookie parsing for refresh tokens
nodemailer	^9.0.3	Email sending
pdfkit	^0.19.1	Certificate PDF generation
pdf-lib	^1.17.1	PDF manipulation
qrcode	^1.5.4	QR code generation for certificates
socket.io	^4.8.3	Real-time WebSocket server
node-cron	^4.6.0	In-process cron job scheduling
pino	^10.3.1	Structured JSON logging
zod	^4.4.3	Input schema validation
axios	^1.18.1	HTTP client (M-Pesa Daraja API)
dotenv	^17.2.3	Environment variable loading

14.2 Client Dependencies

Package	Version	Purpose
react	^19.1.1	UI framework
react-dom	^19.1.1	React DOM renderer
react-router-dom	^7.9.3	Client-side routing
axios	^1.13.2	HTTP client for API calls
tailwindcss	^4.1.13	Utility-first CSS framework

Package	Version	Purpose
socket.io-client	^4.8.3	Real-time WebSocket client
react-toastify	^11.0.5	Toast notifications
sonner	^2.0.7	Additional toast notifications
framer-motion	^12.23.24	Animation library
swiper	^12.0.2	Image/content carousel
jspdf	^3.0.4	Client-side PDF generation
html2canvas-pro	^2.3.0	HTML-to-canvas for PDF export
qrcode.react	^4.2.0	QR code rendering in React
react-icons	^5.5.0	Icon library (Font Awesome, etc.)

15. Known Limitations & Technical Debt

Item	Severity	Description	Recommended Resolution
Database migration in progress	Medium	The system is mid-migration from MongoDB to MySQL. The MySQL schema is complete and the server uses MySQL exclusively. However, the controllers still contain some Mongoose-style code paths (commented or legacy). A full cutover validation pass is recommended.	Complete the cutover documentation in MIGRATION_PLAN.md; run a controller audit
No test suite	Medium	The codebase has no automated tests. Changes carry risk of undetected regressions.	Add integration tests for auth, research lifecycle, and payment flow
M-Pesa in sandbox mode	High	The M-Pesa integration is configured against the Safaricom sandbox. Live payments will fail until production credentials are configured.	Follow Section 7.5 M-Pesa Production Checklist
Single server deployment	Medium	The application runs on a single server. There is no load balancing or high availability.	For higher availability: add a MySQL read replica; consider a second app server behind a load balancer
Cron runs in-process	Low	Background jobs run inside the Node process. If the process crashes and restarts, the next run is at the next scheduled time.	Consider external cron (system crontab or cloud scheduler) for critical jobs
No search on medical tables	Low	Full-text search exists for research and tenders but not for appointments, blood donors, etc.	Add FULLTEXT indexes to frequently-queried text columns

ENV VARIABLE

Client ENV

VITE_API_URL=

Server ENV

NODE_ENV=
PORT=
CORS_ORIGINS=
FRONTEND_URL=
JWT_SECRET=
JWT_REFRESH_SECRET=
JWT_EXPIRES_IN=
JWT_REFRESH_EXPIRES_IN=
DB_HOST=
DB_PORT=
DB_NAME=
DB_USER=
DB_PASSWORD=
DB_SSL=
DB_SSL_REJECT_UNAUTHORIZED=
DB_TIMEZONE=
DB_LOGGING=
DB_POOL_MAX=
DB_POOL_MIN=
DB_POOL_ACQUIRE_MS=
DB_POOL_IDLE_MS=

EMAIL_SERVICE=
EMAIL_USER=
EMAIL_PASSWORD=

MPESA_CONSUMER_KEY=
MPESA_CONSUMER_SECRET=
MPESA_SHORTCODE=
MPESA_PASSKEY=
MPESA_API_URL=
MPESA_CALLBACK_URL=
MPESA_CALLBACK_TOKEN=
MPESA_INITIATOR_NAME=
MPESA_INITIATOR_PASSWORD=
MPESA_CERT_PATH=
MPESA_B2C_CALLBACK_URL=
USE_PRODUCTION_AMOUNTS=

CERTIFICATE_SIGNING_SECRET=

RESEARCH_EXPIRY_CRON_ENABLED=
REVIEWER_REMINDER_CRON_ENABLED=

LOG_LEVEL=
BCRYPT_SALT_ROUNDS=

SEED_SUPERADMIN_EMAIL=
SEED_SUPERADMIN_PASSWORD=